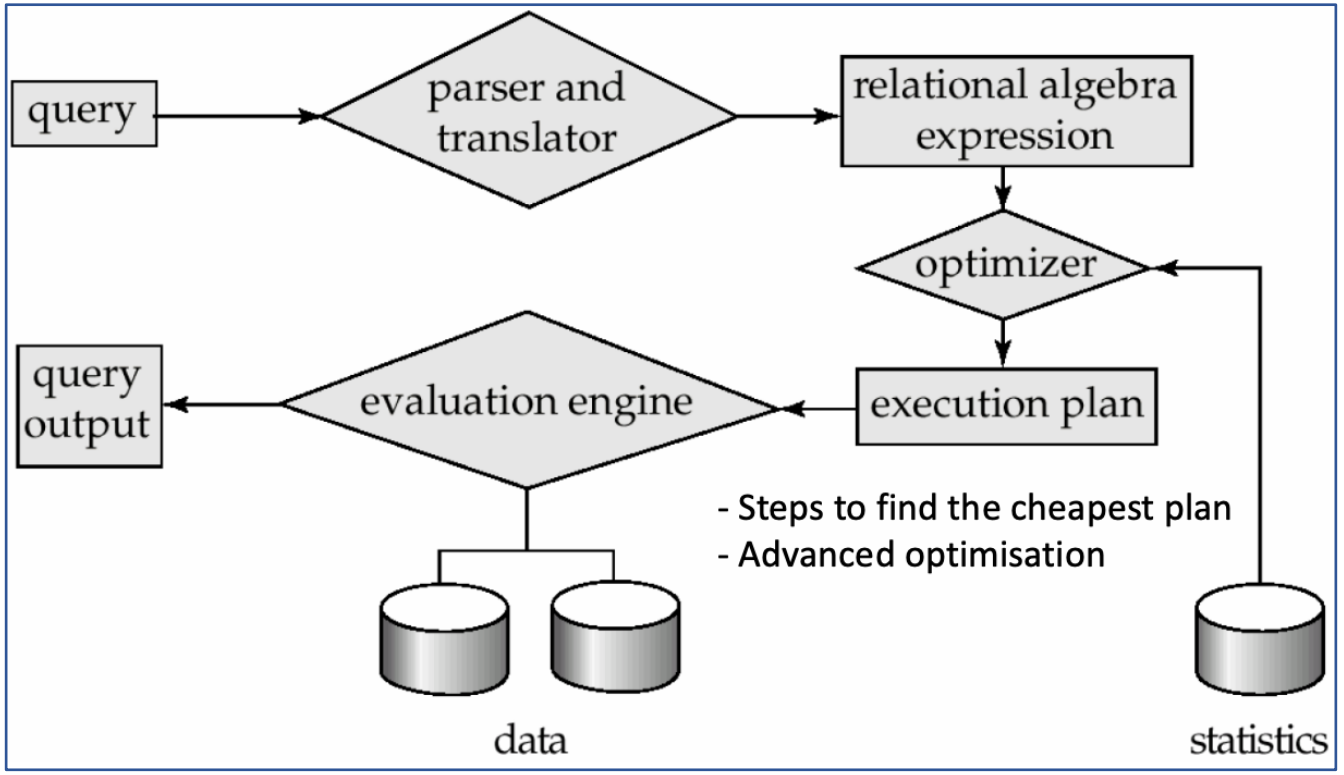
COMP90050 ADBS

Week 2

Q1 Query Optimization Approach



e.g. number of distinct values for an attribute

- **Enumerating all plans**: enumerating all plans and choose the best one based on cost estimation
- **Heuristic**: using a set of rules that typically (but not in all cases) improve execution performance
- **Adaptive plan**: execute a part/some parts of a query plan first to re-evaluate the cost of the other parts of the query plan that haven't been executed yet, to have a better overall cost estimation and hence better plan

Q1 Query Optimization Approach

1. Discuss which query optimisation approach(es) (enumerating all plans, heuristic based, adaptive plans) can be suitable for the following scenarios:

   - **Scenario A**: Given a table with 1 million tuples, run the following query:

```
SELECT customer
FROM Table
WHERE spend BETWEEN 100 AND 200
AND birth_year> 2000;
```

   - **Scenario B**: Given 5 tables with 1 million tuples in each table, run a query:

```
SELECT T1.name, T2.salary, T3.qualification, T4.phone, T5.leader
FROM Table1 T1
    INNER JOIN Table2 T2 ON T2.id = T1.id
    INNER JOIN Table3 T3 ON T3.id = T1.id
    INNER JOIN Table4 T4 ON T4.id = T1.id
    INNER JOIN Table5 T5 ON T5.department = T1.department
WHERE T1.age > 50;
```

Scenario A: heuristic approach can be suitable due to the simplicity of the query and small size of the table.
Adaptive plan CANNOT be used if purely heuristic-based

Scenario B: enumerating approach can be more suitable due to the complexity of the query.
Adaptive plan CAN be used for cost-based query optimizer.

Q2 Performance Improvement

2. A particular query on a table A used to run quite efficiently in a DBMS. After inserting many records and deleting many other records from table A, that same query is now taking more time to run, even when the total number of records has not changed. What can be the reason for that? What can you do as the user/database administrator of that DBMS to improve the performance of this query?

Reason:

1. After insertions and deletions, the statistics of a table may not be updated instantly.
2. As the statistics are used to estimate the cost of a query, wrong statistics are causing wrong cost estimations.
3. Hence the query optimizer is now choosing a query plan that is no longer optimal for that query.

What to do:

Enforcing statistical recompilation option can be used to update the statistics of the table.

Q3

3. When a database needs to join two tables using a page-oriented nested loop join algorithm, both tables are found to be in the main memory. Will it change how the cost of a cost-based optimiser calculates the cost of the query plan?

No, the cost calculation of a query plan will be same as if both tables were on disk.

Q4

4. What are the possible approaches that you can take to improve query performance in practice?

1. Use index if necessary
2. Force a query plan instead of the plan chosen by the optimizer
3. Enforce statistic recompilation
4. Rewrite query with parameters for execution plan reuse
5. Store derived data (when you frequently need derived values and data do not change frequently)
6. Use pre-joined tables (when tables need to be joined frequently).

Q5 Cost and Seeks

5. In the worst case, if there is enough memory only to hold one page/block of each table, what are the estimated cost of simple nested loop join and block nested loop join? What are the number of seeks for simple nested loop join and block nested loop join?

Let the two tables be r and s, where r is outer relation and s is inner relation.

$b_x = \#pages\ in\ table\ x\ ,\ \ n_x = \#tuples\ in\ table\ x$

Simple nested loop join (for each tuple in table r, scan through table s to find matching pairs with the tuple)

$$cost = b_r + (n_r \times b_s)$$
$$seek = b_r + n_r$$

For each page in r:
- seek the page in r
- for each tuple in the page: seek table s

Hence: need to seek each page in r once, and seek s n_r times

Block nested loop join (for each block in table r, scan through table s to find matching pairs with the block)

$$cost = b_r + (b_r \times b_s)$$
$$seek = b_r + b_r = 2b_r$$

For each page in r:
- seek the page in r
- seek table s

Hence: need to seek each page in r once, and seek s b_r times

Others

- Slide: https://github.com/AngieYYF/COMP90050_2026_S1
- Group project team – form team and decide topic by end of this week
- Next week lab: MySQL setup (week 1 tutorial instruction)